

Bitcoin: Funzionamento Informatico

NECKER

10 Novembre 2025

«Non è reale eppure funziona: una complessità a molti invisibile che la rende viva e indomabile da nessuno.»

– NECKER

Indice

1	In Cosa Consiste Bitcoin Nel Concreto	2
2	Block-Chain	3
2.1	concatenazione	3
3	Miners	4
3.1	Struttura Dati di Base	4
3.2	Costruzione Albero Transazioni	5
3.3	Ricerca del PoW	5
3.4	Pubblicazione	6
3.5	Problema Fork	6
3.6	Affidabilità della rete e consenso	7
3.6.1	Regole	7
3.6.2	Incentivi	7
3.6.3	Disincentivi	7
4	Hardware e efficienza	7
4.1	Pool di mining e varianza	8
5	Transazione Bitcoin	9
5.1	Serializzazione Legacy	9
5.2	SegWit (BIP-141)	10
6	Risorse:	11

1 In Cosa Consiste Bitcoin Nel Concreto

Bitcoin, dal punto di vista informatico, non è un file o una stringa di bit clonabile, ma un **registro contabile distribuito** e immutabile. La sua essenza si basa sui seguenti concetti:

- **Natura Immateriale e Decentralizzata:** Essendo una valuta virtuale, non ha un corrispettivo materiale (come oro o riserve statali). Il suo valore è garantito dalla **domanda/offerta** e dalla **scarsità algoritmica** (offerta limitata a 21 milioni di unità) e dalla **sicurezza crittografica** del suo protocollo.
- **Non è una Stringa di Bit Clonabile (Il Problema della Doppia Spesa):**
 - **Il Problema:** Se Bitcoin fosse semplicemente una stringa di bit (come un file digitale), potrebbe essere copiata e spesa più volte (**Double Spending Problem**).
 - **La Soluzione:** Bitcoin risolve questo problema non esistendo come entità fisica, ma come **Storia di Transazioni**. Un Bitcoin è, in sostanza, la prova crittografica e matematicamente verificata di una **UTXO** (Unspent Transaction Output) non ancora spesa, registrata in modo definitivo nella Blockchain.
- **Definizione Concreta: La Blockchain:**
 - Bitcoin consiste in una **Blockchain**: una catena di **Blocchi** contenenti gruppi di **Transazioni** validate.
 - La proprietà di un Bitcoin non è data dal possesso di un oggetto digitale, ma dal possesso di una **Chiave Privata** che permette di firmare crittograficamente (autorizzare) la spesa di una specifica UTXO registrata sulla catena.
- **Infrastruttura Della Rete**
 - **Protocollo peer-to-peer:** Bitcoin è un protocollo P2P che consente a nodi indipendenti di scambiarsi blocchi e transazioni senza intermediari centrali. I nodi comunicano via gossip e validano autonomamente le informazioni ricevute.
 - **Tipi di nodi:**
 - * nodi *full* che mantengono la catena completa (Ledger)
 - * nodi *pruned* che mantengono solo parte della chain per risparmiare spazio
 - * nodi client *light*/SPV che controllano solo prove merkle per le transazioni
 - * La dimensione completa del ledger è di circa 500 GB, mentre un nodo pruned riduce drasticamente lo storage necessario.
 - **Mempool e propagazione:**
 - * Effettuata una transazione, essa entra nel mempool dei nodi in attesa di essere incluse in un blocco, infatti una transazione non è immediata e prima va confermata, cioè inclusa in un blocco (vedremo più avanti che ciò non basterà -> vedi **Problema Fork**).
 - * quando una transazione è nel mempool significa che è stata propagata in broadcast a tutti i nodi della rete. L'ordinamento è spesso basato sulle fee che influenzano la probabilità e i tempi di conferma (la fee è opzionale, funziona perchè generalmente si vuole che la transazione venga approvata il prima possibile).
 - **Ruolo dei miner/validatori:** I miner possono essere chiunque e competono per risolvere il PoW (trattato dopo -> vedi PoW) e proporre blocchi; non sono "autorità" centrali ma partecipanti incentivati (reward + fee). La sicurezza proviene dall'ampio consenso e dalla difficoltà del PoW.

- **Wallets e chiavi:** I portafogli gestiscono coppie di chiavi (pubblica/privata). La custodia della chiave privata equivale al controllo delle UTXO associate: perdere la chiave significa perdere l'accesso ai fondi.
- **Fiducia e consenso:** La fiducia è distribuita: a meno che il 50%+1 della potenza di calcolo non sia controllata da un attore malevolo, la rete rimane sicura. Le regole di consenso sono codificate nel software open-source (es. Bitcoin Core) e sono applicate autonomamente dai nodi (vedi **Affidabilità della rete e consenso**).
- **Infrastruttura ausiliaria:** Oltre ai nodi, esistono servizi come block explorers, wallet provider, exchange e layer secondari (es. Lightning) che migliorano l'usabilità ma possono introdurre punti centrali opzionali.

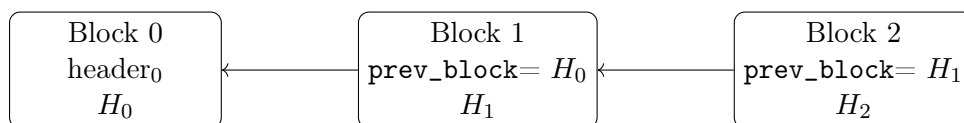
2 Block-Chain

La **blockchain** è una struttura dati che permette la registrazione sicura, trasparente e immutabile di tutte le transazioni effettuate nella rete Bitcoin. Essa è composta da una sequenza di **blocchi**, ognuno dei quali contiene un insieme di transazioni validate, un riferimento al blocco precedente (tramite l'hash) e un **Proof of Work (PoW)** che ne garantisce l'integrità e la sicurezza. La catena parte dal **blocco genesi** (genesis block) creato da Satoshi Nakamoto il 3 gennaio 2009. per verificare se una transazione è valida, deve:

- **Inclusa In Un Blocco Valido:** deve essere inclusa in un blocco che rispetta tutte le regole di consenso della rete.
- **First Block Raggiungibile:** il blocco in cui è inserita la transazione deve essere raggiungibile dalla catena principale e da esso si può risalire al primo blocco (genesis block).

2.1 concatenazione

La **concatenazione** dei blocchi avviene tramite l'inclusione dell'hash del blocco precedente (hash dell'header del blocco **N-1**) nell'header del blocco corrente. Questo crea una catena in cui ogni blocco dipende dal precedente, garantendo l'integrità della catena: modificare un blocco richiederebbe la ricalcolazione del PoW di tutti i blocchi successivi, rendendo praticamente impossibile alterare la storia delle transazioni senza il consenso della maggioranza della rete.



Verifica: per confermare che il collegamento sia valido si ricalcola l'hash dell'header precedente $H_{i-1} = \text{SHA256}(\text{SHA256}(\text{header}_{i-1}))$ e lo si confronta con il valore memorizzato in **prev_block**. Se coincidono, il blocco punta correttamente al precedente.

3 Miners

I "miner" (minatori) sono i partecipanti della rete che mettono potenza di calcolo al servizio della produzione di nuovi blocchi. Il loro lavoro principale è raccogliere transazioni valide dal mempool, costruire un blocco candidato (block template).

3.1 Struttura Dati di Base

Costruzione del block template: il miner seleziona le transazioni dal mempool (a suo piacimento), calcola la Merkle root e crea la transazione coinbase che contiene la ricompensa di blocco (subsidy) e le fee raccolte.

struttura blocco:

Campo	Bytes	Descrizione
Magic	4	0xD9B4BEF9 (mainnet)
Size	4	Dimensione blocco
Header	80	Vedi sotto
Tx Count	1-9 (varint)	Numero transazioni
Tx	variabile	Transazioni raw

- **Magic:** identificatore di rete (mainnet, testnet, regtest), segnala che è un protocollo bitcoin.
- **Size:** dimensione totale del blocco in byte.
- **Header:** 80 byte fissi (**vedi tabella dopo**).
- **Tx Count:** numero di transazioni incluse (varint).
- **Tx:** lista di transazioni effettive (in formato raw).

struttura header:

Offset	Bytes	Campo	Tipo	Note
0	4	Version	int32_t	bit 0-3: versione
4	32	prev_block	char[32]	Hash blocco precedente (LE)
36	32	merkle_root	char[32]	Radice Merkle (LE)
68	4	timestamp	uint32_t	Unix timestamp
72	4	bits	uint32_t	Target compattato
76	4	nonce	uint32_t	Contatore PoW

- **Version:** indica la versione usata del software Bitcoin per creare il blocco.
- **prev_block:** hash del blocco precedente (little-endian).
- **merkle_root:** radice dell'albero delle transazioni incluse.
- **timestamp:** tempo di creazione del blocco (unico valore variabile insieme al nonce, esso infatti può essere variato di poco per provare a trovare il nonce corretto).
- **bits:** rappresenta il numero di zeri nel target di difficoltà in formato compresso (primo byte = esponente + ultimi tre byte = mantissa). Quest valore è uguale per tutti i miner che lavorano sullo stesso blocco, ogni 2016 blocchi (circa ogni 2 settimane) viene ricalcolato in base alla velocità con cui i blocchi precedenti sono stati trovati per mantenere costante il tempo medio di generazione del blocco (circa 10 min).

Formula retarget:

```

nuova = vecchia * (tempo_reale / 1209600)
clampata tra 0.25x e 4x

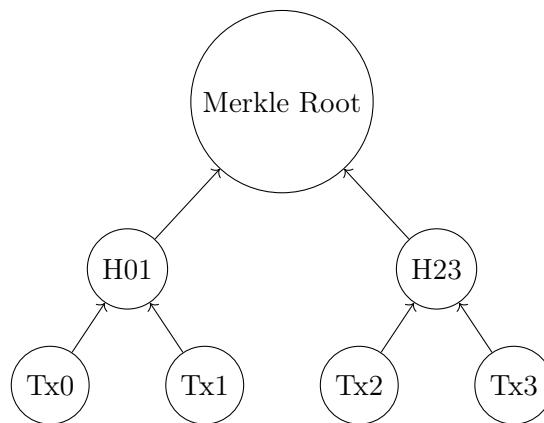
```

- **nonce:** contatore variabile usato per il PoW (numero trovato che se inserito nell'hash, verifica il completamento del puzzle).

3.2 Costruzione Albero Transazioni

Nota: quando si costruisce il blocco, dobbiamo garantire che le transazioni incluse non vengano più modificate (o rimosse) una volta che il blocco è stato pubblicato. Per fare ciò si utilizza una struttura dati chiamata **Merkle Tree**:

- **Merkle Tree:** struttura ad albero binario in cui ogni foglia rappresenta l'hash di una transazione, e ogni nodo interno è l'hash concatenato dei suoi figli. La radice dell'albero (Merkle root) viene inclusa nell'header del blocco per garantire l'integrità di tutti i dati. Se anche una sola transazione viene modificata, l'hash della radice cambierà, invalidando il blocco.



3.3 Ricerca del PoW

Nota: creare la struttura del blocco, dobbiamo scoraggiare utenti malevoli a creare blocchi falsi (vedi **Affidabilità della rete e consenso**). Per questo motivo il miner deve trovare un valore di **nonce** tale che l'hash dell'header del blocco sia inferiore al target di difficoltà specificato nel campo **bits**. Questo processo è noto come **Proof of Work (PoW)**:

- **Double SHA-256:** il PoW richiede il calcolo del double-SHA256 sull'header del blocco. Il miner varia il campo **nonce** (e, se necessario, l'il **timestamp** di qualche secondo/minuto) e ripete il calcolo fino a trovare un hash che soddisfi il target (trovare un hash con almeno tot bit iniziali a 0 dettati dal campo **bits**).

$$H := \text{SHA256}(\text{SHA256}(\text{header}))$$

dove **header** è la concatenazione dei campi dell'header del blocco (|textbf:: indica simbolo di concatenazione):

$$\text{header} = \text{Version} :: \text{prev_block} :: \text{merkle_root} :: \text{timestamp} :: \text{bits} :: \text{nonce}$$

Interpretando l'hash come un intero non negativo a 256 bit H_{int} , la condizione di validità del PoW è:

$$H_{\text{int}} \leq \text{target}$$

Il campo compatto `bits` codifica il target secondo la formula (E = esponente, C = coefficiente a 3 byte):

$$\text{target} = C \times 2^{8(E-3)}$$

La difficoltà si definisce come

$$\text{difficulty} = \frac{\text{target}_{\max}}{\text{target}}, \quad \text{target}_{\max} = 0xFFFF \times 2^{8(0x1d-3)}$$

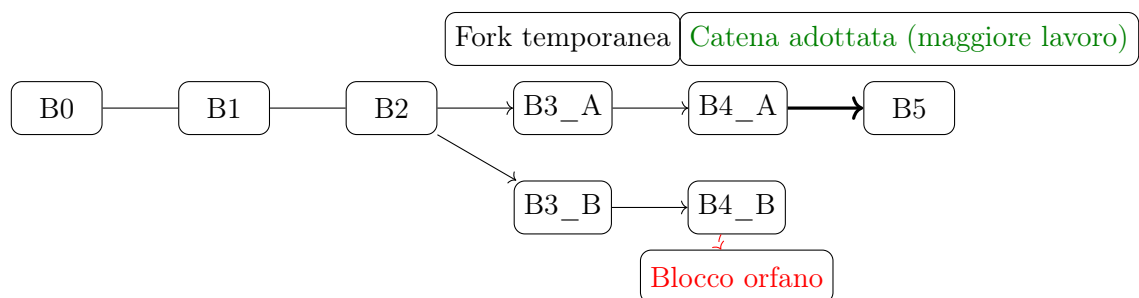
3.4 Pubblicazione

Nota: quando un blocco viene pubblicato, chi decide se è valido o meno sono i nodi della rete che lo ricevono (quindi altri miner). Se il blocco soddisfa tutte le regole di consenso (formato corretto, PoW valido, transazioni valide, nessuna double-spend, ecc.), viene aggiunto alla catena locale del nodo. La regola che determina quale blocco viene aggiunto è quella di: chi prima trova il valore vince, se più miner trovano il numero insieme (vedi **Problema fork**).

- **Verifica:** i nodi verificano il PoW, la validità delle transazioni e l'integrità del blocco.
- **Aggiunta alla catena:** se valido, il blocco viene aggiunto alla blockchain locale del nodo. per far sì che un nodo sia convalidato serve che almeno il 51% della potenza di calcolo totale della rete accetti il blocco.
- **Propagazione:** il blocco viene propagato in broadcast ad altri nodi della rete.

3.5 Problema Fork

Nota: quando un blocco viene pubblicato, non arriva a tutti istantaneamente, se in un lasso di tempo molto breve due miner trovano un blocco valido quasi simultaneamente (un miner in Cina e uno in Messico), si crea una **fork temporanea** della catena. I nodi che ricevono i blocchi in momenti diversi potrebbero avere opinioni divergenti su quale sia l'ultimo blocco valido. La risoluzione avviene quando un miner trova il blocco successivo, estendendo una delle due catene; la rete adotta la catena più lunga (con maggiore lavoro cumulativo) come valida, e i blocchi dell'altra catena diventano orfani.



- **periodo di attesa conferme:** per mitigare il rischio di fork, gli utenti attendono più conferme (blocchi successivi) prima di considerare una transazione definitiva (tipicamente 6 conferme).
- **periodo del reward:** per lo stesso motivo, la ricompensa del blocco ($\text{subsidy} + \text{fee}$) viene considerata definitiva solo dopo 100 blocchi (circa 16 ore), per evitare che un miner possa spendere la ricompensa di un blocco che poi diventa orfano.

3.6 Affidabilità della rete e consenso

Nota: dato che non esiste un certificatore centrale che garantisca l'integrità della rete, Bitcoin si affida a un meccanismo di consenso distribuito basato su regole rigorose e incentivi economici per garantire che tutti i nodi concordino sullo stato della blockchain. Idealmente l'unico modo per un attaccante di compromettere la rete è controllare più del 50% +1 della potenza di calcolo totale (**Geopoliticamente il problema esiste già dato che in Cina è allocata più del 50% della potenza di calcolo**).

3.6.1 Regole

- **No Double-Spending:** per garantire che una transazione non venga spesa due volte, ogni transazione deve fare riferimento a UTXO non ancora spesi.
- **Validità delle transazioni:** ogni transazione deve essere firmata con la chiave privata corrispondente all'UTXO di input, e deve rispettare le regole di scripting (ScriptPubKey e ScriptSig).
- **Scarto Transazioni:** alla convalida di un blocco, nel calcolo del prossimo blocco da minare, i nodi scartano tutte le transazioni già incluse in blocchi precedenti.
- **convalidazione:** se esistono più blocchi concorrenti, i nodi adottano la catena con il maggior lavoro cumulativo (catena più lunga).

3.6.2 Incentivi

per incentivare i miner a partecipare onestamente al processo di consenso, Bitcoin utilizza un sistema di ricompense:

- **Block Reward:** ogni blocco valido minato conferisce al miner una ricompensa fissa (subsidy) che si dimezza ogni 210.000 blocchi (circa ogni 4 anni) in un evento chiamato "halving" (ora siamo a circa 3.125 BTC a blocco minato).
- **Transaction Fees:** oltre alla ricompensa di blocco, i miner raccolgono le fee associate alle transazioni incluse nel blocco. Le fee sono pagate dagli utenti per incentivare i miner a includere le loro transazioni più rapidamente.

3.6.3 Disincentivi

- **Costi Operativi:** il mining richiede hardware specializzato e un consumo energetico significativo. Questi costi operativi fungono da deterrente contro attacchi economici, poiché un attaccante dovrebbe sostenere costi elevati per controllare una porzione significativa della potenza di calcolo della rete.

4 Hardware e efficienza

- **Evoluzione:** CPU → GPU → FPGA → ASIC specializzati per SHA-256. Oggi il mining è dominato da ASIC progettati per massimizzare throughput e efficienza energetica.
- **Metriche:** consumo energetico (Joule) per unità di lavoro (TH/s) e costo per hash influenzano la redditività.

4.1 Pool di mining e varianza

- **Perché i pool:** il mining ha alta varianza (trovare un blocco è raro ma ricompensante) quindi i miner si uniscono in *pool* per stabilizzare i ricavi. Il pool aggrega la potenza di calcolo e ripartisce le entrate tra i partecipanti secondo regole prefissate.
- **Schemi di remunerazione:** i pagamenti ai partecipanti possono seguire diversi schemi, i più comuni sono:
 - **PPS (Pay Per Share):** il miner riceve un pagamento fisso per ogni *share* valida inviata al pool, indipendentemente dal fatto che il pool trovi effettivamente un blocco. Riduce al massimo la varianza, ma trasferisce il rischio al pool operator.
 - **PPLNS (Pay Per Last N Shares):** il pagamento è proporzionale alle ultime N share inviate; questo schema premia i miner che rimangono fedeli al pool e riduce la possibilità di "pool hopping".
- **Share:** una *share* è una prova di lavoro a difficoltà ridotta (target imposto dal pool) che dimostra che un miner ha contribuito con lavoro utile. Le share non sono blocchi validi per la rete ma servono al pool per contabilizzare il contributo di ogni miner.
- **Ruolo del pool operator:** il pool coordina i job di mining (assegna header/nonce ranges), raccoglie le share, invia il blocco risultante alla rete e gestisce la distribuzione delle ricompense. Questo introduce un punto di coordinamento (potenziale centralizzazione) e richiede fiducia nel corretto pagamento da parte dell'operator.
- **Pagamento anche senza blocco trovato:** alcuni schemi (es. PPS) forniscono pagamenti regolari anche se il pool non trova blocchi, in pratica il pool assorbe il rischio statistico e lo distribuisce tra i partecipanti.
- **Stratum Protocol:** il protocollo standard usato tra miner e pool per distribuire lavori (job), inviare share e ridurre overhead di comunicazione. Stratum è leggero, efficiente e permette lavori aggiornati frequentemente.

5 Transazione Bitcoin

5.1 Serializzazione Legacy

Le transazioni Bitcoin sono serializzate in un formato binario specifico che consente la trasmissione efficiente attraverso la rete. La struttura di una transazione Bitcoin include vari campi chiave, tra cui version, input, output e locktime. Di seguito è riportata la struttura dettagliata di una transazione Bitcoin in formato raw legacy (pre-SegWit ma ancora ampiamente utilizzato):

Inoltre ricordiamo che la valuta è effettivamente la transazione stessa, quindi un utente possiede bitcoin se possiede le transazioni che lo assegnano a lui (UTXO).

Tabella 1: Formato raw transazione (legacy)

Campo	Bytes	Descrizione
Version	4	Versione (little-endian)
Input Count	varint	Numero di input
Input[i]:	variable	Voci degli input
- Prev Hash	32	Hash della tx precedente
- Prev Index	4	Indice dell'output speso
- ScriptSig Len	varint	Lunghezza dello ScriptSig
- ScriptSig	varint	ScriptSig
- Sequence	4	Sequence
Output Count	varint	Numero di output
Output[j]:	variable	Voci degli output
- Value	8	Valore in satoshi
- ScriptPK Len	varint	Lunghezza dello scriptPubKey
- ScriptPK	varint	ScriptPubKey
LockTime	4	Lock time del blocco/transazione

- **Version:** indica la versione della transazione.
- **Input Count:** numero di input nella transazione espresso in varint (es: se devo spendere **5 BTC** e ho 3 transazioni ciascuna dal valore di: **1 BTC**, **1.5 BTC**, **3 BTC**, per effettuare la transazione le devo includere tutte e 3).
- **Input[i]:**
 - **Prev Hash:** hash della transazione precedente da cui si sta spendendo l'output.
 - **Prev Index:** indice dell'output specifico nella transazione precedente.
 - **ScriptSig Len:** lunghezza dello ScriptSig.
 - **ScriptSig:** script che fornisce la prova di proprietà dell'output (firma digitale).
 - **Sequence:** campo usato per funzionalità avanzate come RBF (Replace-By-Fee).
- **Output Count:** numero di output nella transazione espresso in varint (es: se voglio inviare **4 BTC** a un destinatario e ricevere **1 BTC** come resto, avrò 2 output, se includo la fee avrò anche l'output della fee).
- **Output[j]:**
 - **Value:** valore dell'output in satoshi (1 BTC = 100.000.000 satoshi).
 - **ScriptPubKey Len:** lunghezza dello ScriptPubKey.
 - **ScriptPubKey:** script che specifica le condizioni per spendere l'output (es. indirizzo del destinatario).
- **LockTime:** specifica il tempo o il blocco minimo prima che la transazione possa essere inclusa in un blocco.

5.2 SegWit (BIP-141)

Segregated Witness (SegWit) è un aggiornamento del protocollo Bitcoin introdotto con il BIP-141 che modifica il formato delle transazioni per separare i dati di firma (witness) dal resto della transazione. Questo cambiamento consente di aumentare la capacità delle transazioni, migliorare la scalabilità e risolvere alcuni problemi legati alla malleabilità delle transazioni.

Tabella 2: Formato raw transazione (SegWit – BIP-141)

Campo	Bytes	Descrizione
Version	4	Versione (little-endian)
Marker	1	(es: 0x00)
Flag	1	(es: 0x01)
TxIn Count	varint	Numero di input
TxIn[i]:	variable	Input
TxOut Count	varint	Numero di output
TxOut[j]:	variable	Output
Witness	variable	Dati witness
LockTime	4	Lock time blocco/trans..

- **marker = 0x00:** byte posto immediatamente dopo il campo **Version**. Non rappresenta una versione ma funge da *marker* che segnala l'inizio del formato SegWit e la presenza del successivo **flag** (cioè indica che la transazione contiene dati witness separati).
- **flag = 0x01:** byte posto subito dopo il **marker** che indica la presenza di dati witness, cioè firme segregate.
- **TxIn Count:** come nel formato legacy, indicano il numero di input della transazione.
- **TxIn[i]:**
 - **Prev Hash:** hash della transazione precedente da cui si sta spendendo l'output.
 - **Prev Index:** indice dell'output
 - **ScriptSig Len:** lunghezza dello ScriptSig (spesso vuoto in SegWit).
 - **ScriptSig:** script che fornisce la prova di proprietà dell'output (spesso vuoto in SegWit).
 - **Sequence:** campo usato per funzionalità avanzate come RBF (Replace-By-Fee).
- **TxOut Count:** come nel formato legacy, indicano il numero di output della transazione.
- **TxOut[j]:**
 - **Value:** valore dell'output in satoshi (1 BTC = 100.000.000 satoshi).
 - **ScriptPubKey Len:** lunghezza dello ScriptPubKey.
 - **ScriptPubKey:** script che specifica le condizioni per spendere l'output (es. indirizzo del destinatario).
- **witness:** separato dopo tutte le tx (contiene firme digitali del mittente e le corrispondenti chiavi pubbliche per verificarle).
- **Peso (Weight):** misura introdotta da SegWit per limitare la dimensione effettiva di un blocco tenendo conto separatamente dei dati witness. Si calcola come:

$$\text{weight}(\text{tx}) = 4 \cdot \text{stripped_size}(\text{tx}) + \text{witness_size}(\text{tx})$$

Dove `stripped_size` è la dimensione della transazione senza i dati witness (equivalente al "legacy" size) e `witness_size = total_size - stripped_size`. Il limite del blocco è espresso in "weight units" (WU):

$$\sum_{tx \in \text{block}} \text{weight}(tx) \leq 4\,000\,000 \text{ WU}$$

Si definisce inoltre la "virtual size" (`vsize`), utile per confrontare transazioni legacy e SegWit:

$$\text{vsize} = \left\lceil \frac{\text{weight}}{4} \right\rceil$$

In pratica SegWit conta i byte non-witness quattro volte più dei byte witness; questo incentiva l'adozione di witness più compatti e permette pacchetti con dati witness grandi senza penalizzare eccessivamente il limite in `vsize`.

6 Risorse:

- Paper di Satoshi Nakamoto (Whitepaper): <https://bitcoin.org/bitcoin.pdf>
- Bitcoin Core (repo): <https://github.com/bitcoin/bitcoin>
- Explorer / blocchi in tempo reale:
 - Mempool.space (visualizzazione mempool e blocchi in tempo reale): <https://mempool.space>
 - Blockstream Explorer: <https://blockstream.info>
 - Blockchair (multi-chain search & analytics): <https://blockchair.com/bitcoin>
 - Blockchain.com Explorer: <https://www.blockchain.com/explorer>
- Siti utili per sviluppatori e documentazione:
 - Bitcoin Developer Guide: <https://developer.bitcoin.org/devguide/>
 - Bitcoin Optech (best practices & guide): <https://bitcoinops.org>
 - BIP (Bitcoin Improvement Proposals) repository: <https://github.com/bitcoin/bips>
 - Bitcoin Stack Exchange (Q&A): <https://bitcoin.stackexchange.com>